

SOLUCIÓN QUIZ 2: PROGRAMACIÓN DINÁMICA

PROFESOR: Federico Melo Barrero

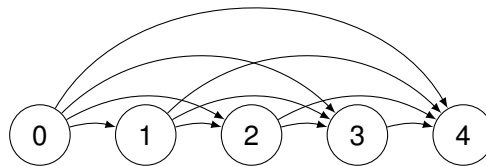
1. Sea $R(m)$ el ingreso máximo que es posible obtener al partir un bloque de duración m en pautas, para $m \in \{0, 1, \dots, n\}$.

$$R(m) = \begin{cases} 0 & \text{si } m = 0 \\ \max_{1 \leq i \leq m} (\text{tarifa}[i] + R(m - i)) & \text{si } m > 0 \end{cases}$$

Símbolo por símbolo:

- $R(m)$: el ingreso máximo que se puede obtener al partir un bloque de duración m en pautas.
 - $R(n)$: el ingreso máximo que se puede obtener al partir un bloque de duración n en pautas, es decir, la respuesta al problema.
 - m : la duración del bloque que falta por repartir en pautas, entre 0 y n .
 - i : la duración de la primera pauta que se corta del bloque de tamaño m , entre 1 y m .
 - $\text{tarifa}[i]$: el precio de una pauta de duración i (esto no es una variable).
 - $\max$: prueba, para el bloque de tamaño m , cada duración posible i de la primera pauta que se corta, y le suma el ingreso óptimo de repartir lo que queda ($m - i$). La mejor de esas m opciones es $R(m)$.
2. Una tabulación de R tiene una complejidad espacial de $\Theta(n)$: cada solución a un subproblema ocupa una entrada del arreglo.

Examinando la recurrencia, es evidente que al computar $R(m)$ con $m > 0$ se utilizan las soluciones de $R(0), \dots, R(m - 1)$. El grafo de dependencias es entonces el siguiente:



Grafo de dependencias de $R(m)$ para $n = 4$.

Se necesitan todos los valores anteriores, por ende, no es posible optimizar el espacio por debajo de $\Theta(n)$, independientemente de si la implementación es top-down o bottom-up.

También utilizando el grafo, es claro que resolver cada uno de los n estados cuesta hasta $\Theta(n)$ trabajo (recorrer todos los vértices anteriores en el grafo, o recorrer el $\max$ en la relación de recurrencia). Ergo, la complejidad temporal es $\Theta(n^2)$: $\Theta(n)$ estados, cada uno con costo de $\Theta(n)$.

3. Implementaciones top-down y bottom up en Python. Ambas con complejidad espacial $\Theta(n)$ y complejidad temporal $\Theta(n^2)$.

```
1 def max_ingreso_top_down(tarifa: list[int]) -> int:
2     n = len(tarifa) - 1
3     ingreso = [None] * (n + 1)
4     ingreso[0] = 0
5
6     return max_ingreso_memo(n, tarifa, ingreso)
7
8 def max_ingreso_memo(m: int, tarifa: list[int], ingreso: list[int]) -> int:
9     if ingreso[m] is not None:
10         return ingreso[m]
11
12     mejor = 0
13     for i in range(1, m + 1):
14         mejor = max(mejor, tarifa[i] + max_ingreso_memo(m - i, tarifa, ingreso))
15
16     ingreso[m] = mejor
17     return mejor
```

```
1 def max_ingreso_bottom_up(tarifa: list[int]) -> int:
2     n = len(tarifa) - 1
3     ingreso = [0] * (n + 1)
4
5     for m in range(1, n + 1):
6         mejor = 0
7         for i in range(1, m + 1):
8             mejor = max(mejor, tarifa[i] + ingreso[m - i])
9         ingreso[m] = mejor
10
11     return ingreso[n]
```